

School of Computer Science and Artificial Intelligence

Lab Assignment - 7.5

Name of Student : Chatharaboina sagar
Enrollment No. : 2303A51522
Batch No. : 22

Task:1 Analyze given code where a mutable default argument causes unexpected behavior. Use AI to fix it.

Bug: Mutable default argument

```
def add_item(item, items=[]):
```

```
    items.append(item)
```

```
    return items
```

```
print(add_item(1))
```

```
print(add_item(2))
```

Expected Output: Corrected function avoids shared list bug.

Prompt: by the above code corrected function avoids shared list bug

Code:

```
lab6.py > ...
1  #--Task--1
2  '''def add_item(item, items=[]):
3      items.append(item)
4      return items
5  print(add_item(1))
6  print(add_item(2))'''
7  #by the above code corrected funtion avoids shared list bug
8  def add_item(item, items=None):
9      if items is None:
10         items = []
11         items.append(item)
12         return items
13  print(add_item(1))
14  print(add_item(2))
15  #--Task--2
16
```

Output:

```
[1]
[2]
```

Task—2: Analyze given code where floating-point comparison fails. Use AI to correct with tolerance.

Bug: Floating point precision issue
def check_sum():
 return (0.1 + 0.2) == 0.3
print(check_sum())
Expected Output: Corrected function

Prompt: by the above checksum program corrected to avoid floating point precision issue

Code:

```
5  #--Task--2
6  '''def check_sum():
7      return (0.1 + 0.2) == 0.3
8  print(check_sum())'''
9  #by tha above check_sum program corrected to avoid floating point precision issue
10 def check_sum():
11     return abs((0.1 + 0.2) - 0.3) < 1e-9
12 print(check_sum())
```

Output:

True

Task-3:

analyse given code where recursion runs infinitely due to missing base case. Use AI to fix.

Bug: No base case

```
def countdown(n):
    print(n)
    return countdown(n-1)
countdown(5)
```

Expected Output : Correct recursion with stopping condition.

Prompt:

then the above code Correct recursion with stopping condition

Code:

```
#Task--3
'''def countdown(n):
    print(n)
    return countdown(n-1)
countdown(5)'''
# then the above code Correct recursion with stopping condition
def countdown(n):
    if n <= 0:
        print("Blast off!")
    else:
        print(n)
        countdown(n-1)
countdown(5)
```

Output:

```
on.exe "c:/sagar/pack1/OneDrive/3-2/AI Assistant Coding/lab6.py"
5
4
3
2
1
Blast off!
PS C:\sagar\pack1\OneDrive\3-2\AI Assistant Coding> █
```

Task—4

Task: Analyze given code where a missing dictionary key causes error. Use AI to fix it.

Bug: Accessing non-existing key

```
def get_value():
    data = {"a": 1, "b": 2}
    return data["c"]
print(get_value())
```

Expected Output: Corrected with .get() or error handling.

Prompt:

then the above code Corrected with .get() or error handling.

Code:

```

38  #--Task--4
39  '''def get_value():
40      data = {"a": 1, "b": 2}
41      return data["c"]
42  print(get_value())'''
43  # then the above code Corrected with .get() or error handling.
44  def get_value():
45      data = {"a": 1, "b": 2}
46      return data.get("c", "Key not found")
47  print(get_value())

```

Output:

```
Key not found
```

Task—5

: Analyze given code where loop never ends. Use AI to detect and fix it.

Bug: Infinite loop

```
def loop_example():
```

```
    i = 0
```

```
    while i < 5:
```

```
        print(i)
```

Expected Output: Corrected loop increments i.

Prompt:

above code corrected infinite loop with incrementing i

Code:

```

50  #--Task--5
51  '''def loop_example():
52      i = 0
53      while i < 5:
54          print(i)'''
55  #by the above code corrected infinite loop with incrementing i
56  def loop_example():
57      i = 0
58      while i < 5:
59          print(i)
60          i += 1
61  loop_example()

```

```

on.exe C:/Sagar/pack1/onedrive/3 2/AI Assistant Coding/
0
1
2
3
4

```

Task—6:

Task: Analyze given code where tuple unpacking fails. Use AI to fix it.

Bug: Wrong unpacking

a, b = (1, 2, 3)

Expected Output: Correct unpacking or using _ for extra values.

Prompt:

Correct unpacking or using _ for extra values

CODE:

```

63  #--Task--6
64  '''a, b = (1, 2, 3)'''
65  #by the above code Correct unpacking or using _ for extra values
66  a, b, _ = (1, 2, 3)
67  print(a, b)

```

Output:

```
1 2
```

Task—7

Task: Analyze given code where mixed indentation breaks execution. Use AI to fix it.

Bug: Mixed indentation

def func():

x = 5

y = 10

return x+y

Expected Output : Consistent indentation applied.

Prompt:

above code corrected with return statement

Code:

```
69  #--Task--7
70  def func():
71      x = 5
72      y = 10
73      return x+y
74  #by the above code corrected with return statement
75  result = func()
76  print(result)
```

Output:

15

Task—8:

Task: Analyze given code with incorrect import. Use AI to fix.

Bug: Wrong import

import maths

print(maths.sqrt(16))

Expected Output: Corrected to import math

Prompt:

Code:

```
78  #--Task--8
79  '''import maths
80  print(maths.sqrt(16))'''
81  #by the above code corrected with correct module name
82  import math
83  print(math.sqrt(16))
84
```

Output:

4.0